

StringEqualityVerifier

May 21, 2026

0.1 String Equality Verifier - Implementazione e Verifica di un esempio pratico

In questo laboratorio, considerato un file a di 30 bit e un **numero primo** casuale $p, 2 \leq p \leq 900$, e' richiesto di determinare tutti i possibili file b per i quali, fissato p , l'output di `MCStringEqualityVerifier(a,b)` e' **input probabilmente uguali**.

Classicamente se volessimo verificare se due file, entrambi di lunghezza ℓ , sono identici dovremmo spedire e confrontare tutti gli ℓ bit. Il protocollo presentato invece funziona valutandone solo $2\ln \ell$. Per far cio' e' necessario introdurre il concetto di **Impronta digitale** di un file.

Considerato un **numero primo** $p, 2 \leq p \leq \ell^2$ e $n \in \mathbb{N}, n < 2^\ell$, definiamo l'**Impronta digitale** di n :

$$f(n) = n \pmod{p}$$

Questa avra' una dimensione ridotta ad al piu' $2\ln \ell$ bit, derivata dal numero di bit necessari a codificare p in binario. Si avra' che $p \leq \ell^2, f(n) < p \rightarrow f(n) \leq \ell^2$ ossia, il **resto** del modulo di p e' strettamente minore di p stesso e quindi al piu' lungo ℓ^2 .

Calcolata l'**Impronta digitale** di ciascuno dei due file, allora bastera' confrontarle fra loro per ottenere il verdetto.

Presento ora l'algoritmo `MCStringEqualityVerifier(a, b)` che prende in *Input* a, b **stringhe binarie** di lunghezza ℓ e restituisce in *output* **input probabilmente uguali** se le **impronte digitali** combaciano, altrimenti **input diversi**.

1. campiona p , primo, uniformemente a caso in $\{2, \dots, \ell^2\}$
2. $f(a) \leftarrow a \pmod{p}$
3. $f(b) \leftarrow b \pmod{p}$
4. if $|f(a) - f(b)| \leq 0 \pmod{p}$
 return **input probabilmente uguali**
5. else
 return **input diversi**

Il protocollo si compone delle seguenti fasi:

- 1) **Alice** seleziona un **numero primo** p a caso fra 2 e ℓ^2
- 2) **Alice** calcola l'**impronta** $f(a) = a \pmod{p}$ del suo file **A** (lungo ℓ bit) e *spedisce* sia il **numero primo** che l'**impronta digitale** a Bob.
- 3) **Bob** riceve p e $f(a)$ e calcola l'**impronta digitale** del suo file, $f(b) = b \pmod{p}$
- 4) **Bob** confronta le due **impronte digitali** (i due resti), e *verifica* se sono uguali.

I due file a questo punto vengono considerati **probabilmente identici**. La parola **probabilmente**, come sempre sta ad indicare la possibilita' d'errore dell'algoritmo, infatti se p , scelto casualmente, e' un **divisore esatto** di $|f(a) - f(b)|$, avremmo comunque che $f(a) - f(b) \pmod{p} = 0$. La probabilita' d'errore e' calcolabile.

Infatti, il numero di **fattori primi** di un numero $n < 2^\ell, n \in \mathbb{N}$ e' al piu' ℓ e, poiche' $p \leq \ell^2$, avremo un totale di numeri primi *possibili* pari a $\pi(p) = \frac{\ell^2}{2 \ln \ell}$ quindi:

$$\mathbb{P}(\ell) \approx \frac{\ell}{\pi(p)} \approx \frac{2 \ln \ell}{\ell}$$

Consideriamo ora un file a di 30 bit, e p primo **fissato**, $2 \leq p \leq 900$.

Per determinare tutti i file per i quali l'output e' **probabilmente uguali** devo calcolare il numero di casi in cui il verdetto dell'algoritmo e' positivo.

Ricordando che un falso positivo avviene se $f(a) \equiv f(b) \pmod{p}$, cioe' quando p e' un **fattore primo** della differenza, $a - b = k \cdot p, k \in \mathbb{Z}$. Inoltre $a, b < 2^{30}$. Infine avremo:

$$B_{\text{probabilmente uguali}} = \{b \in \mathbb{N} \mid b = k \cdot p + a, 0 \leq b < 2^{30}\}$$

Quindi su un massimo $b = 2^{30} \approx 10^9$, tutti i file possibili da 30 *bit*, scelto a *caso* $p = 101$, il numero di file uguali ad a e' pari a $\frac{10^9}{101} \approx 10^7$.

Considerato il totale di numeri primi a disposizione $\pi(900) = \frac{900}{\ln(900)} \approx 132$ La probabilita' che tale valore sia stato scelto da un altro mio collega e' $\mathbb{P}(\text{same}) = \frac{1}{132} \approx 0.76\%$.

Baldini Filippo - 6393212